

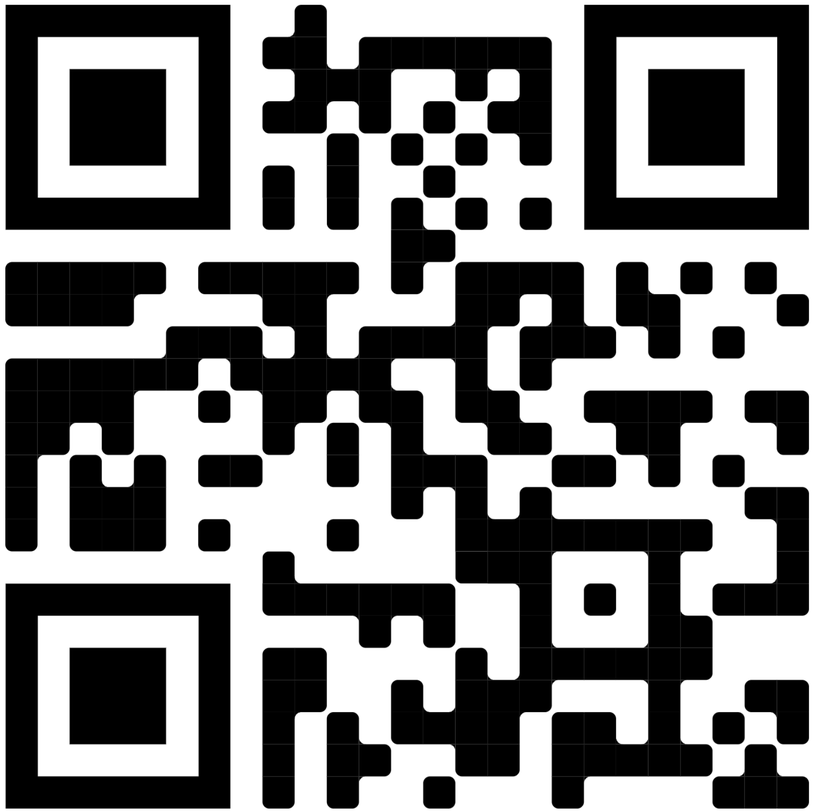


Distributed systems

Lectures: doc. dr. sc. Nikola Tanković

Broadcasting and delivery order



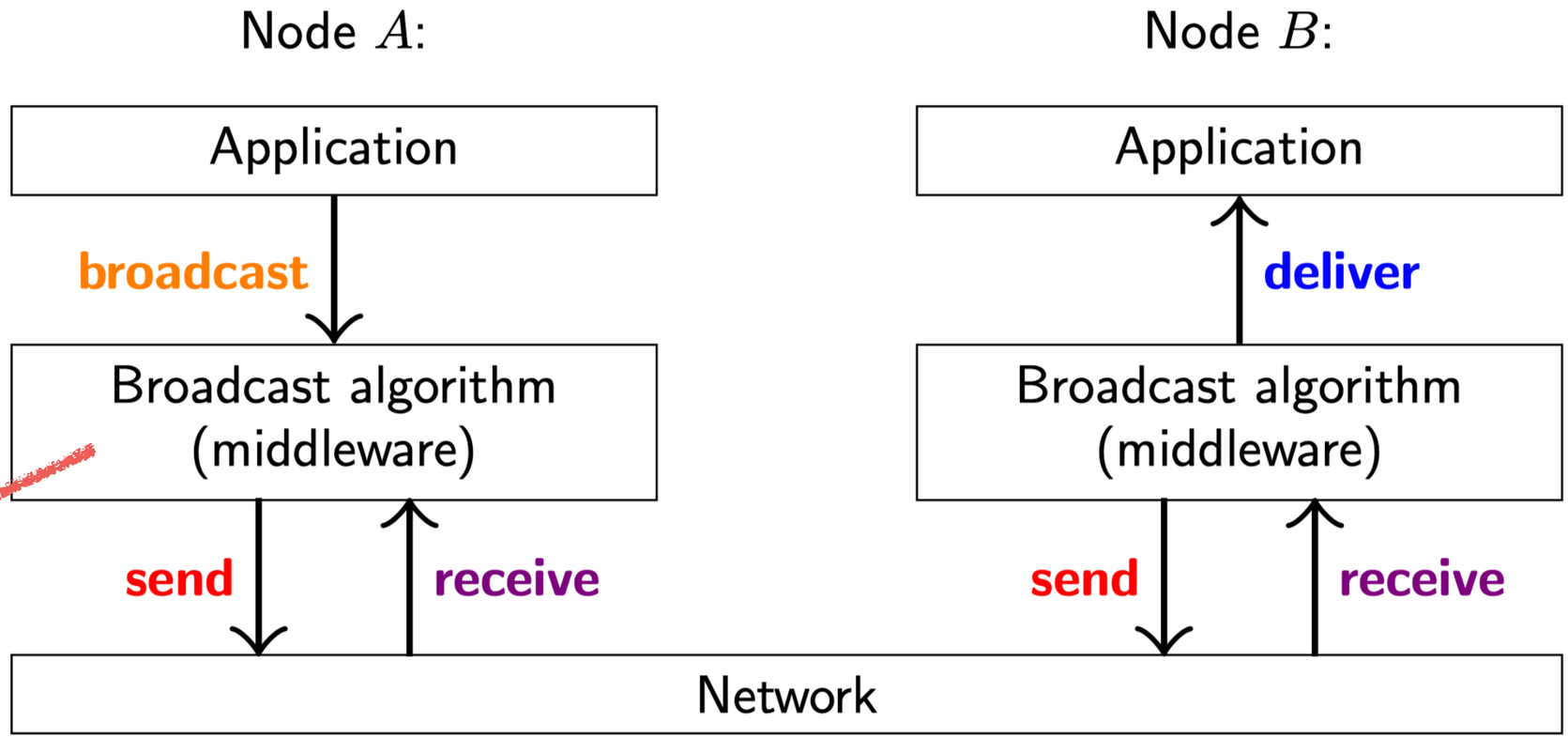


http://bit.ly/rs-tu-sam

Broadcasting

- Broadcast (multicast) is **group communication**
  - One **sender**, all in a group, **deliver** and **receive**
  - No. of members can be **static** (fixed-size) or **dynamic**
  - If a node is faulty, group members carry on
  - Since the Internet (and many networks) support only **unicast,** broadcasting takes that into consideration
- Can be **best-effort** (messages can be dropped) or **reliable** (with re-transmitting)

Receiving and delivering

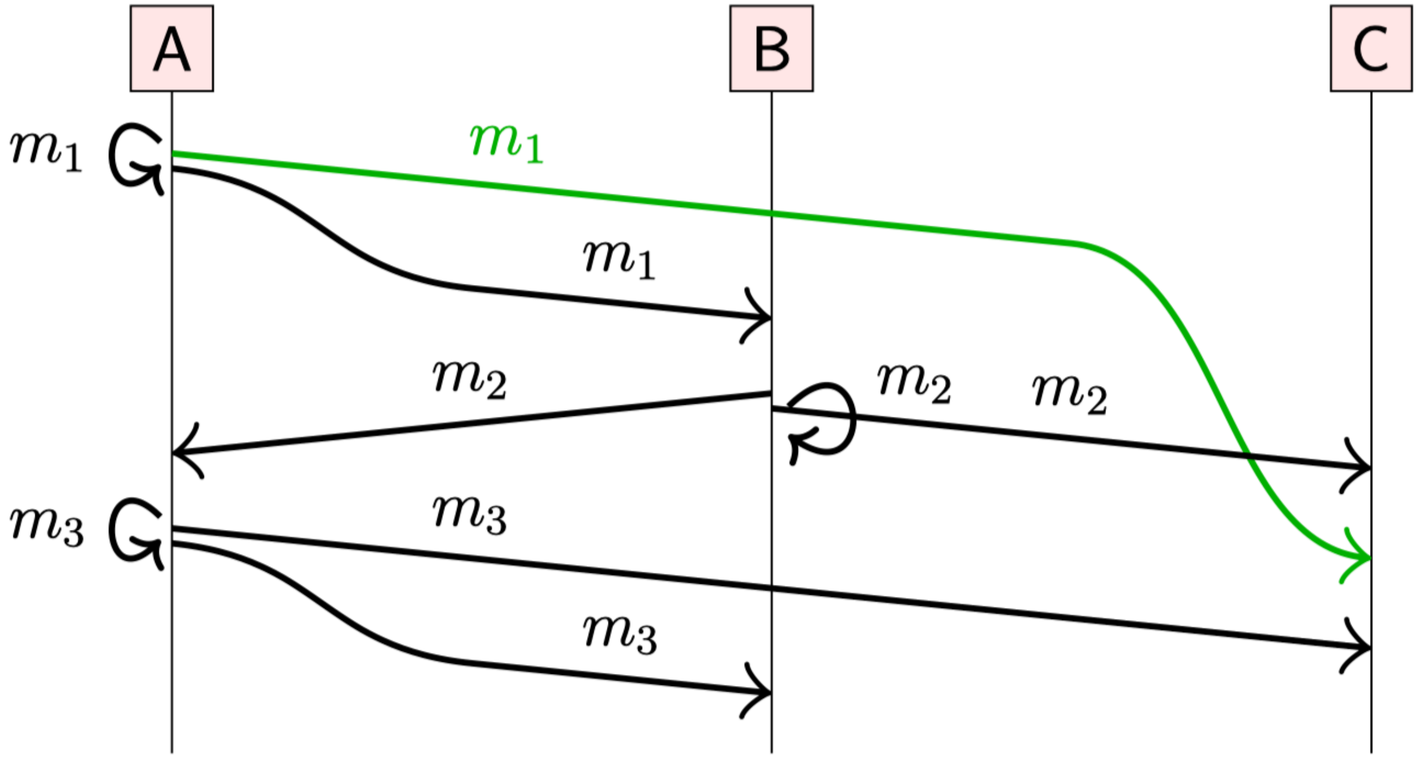


May send to many nodes if on unicast network

Reliable broadcast types

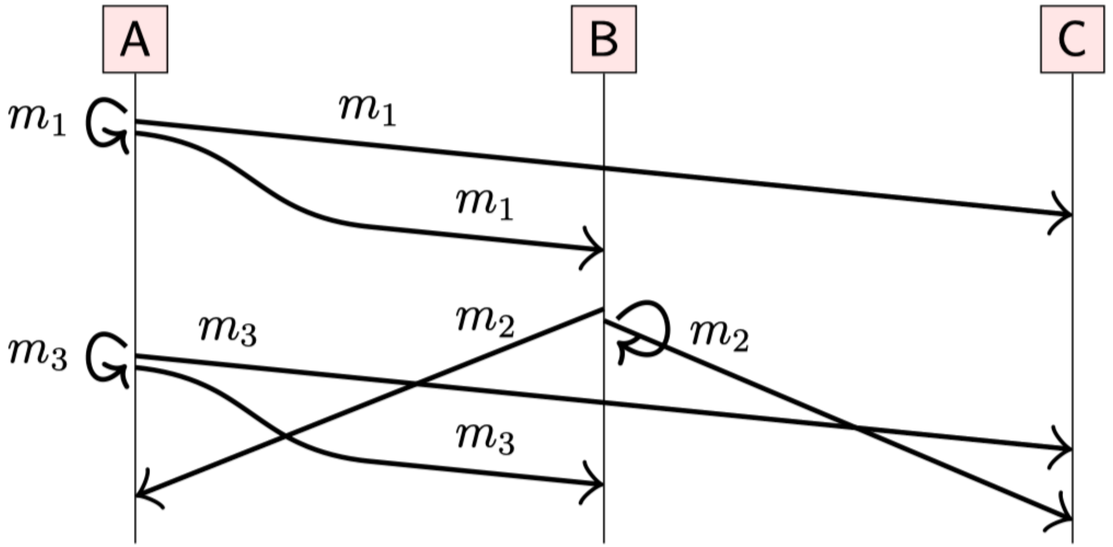
- **FIFO broadcast**
  - Given m_1 and m_2 are delivered **from the same node**, and broadcast(m_1) $\rightarrow$ broadcast(m_2) then m_1 is to be delivered **before** m_2
- **Causal broadcast**
  - If broadcast(m_1) $\rightarrow$ broadcast(m_2) then m_1 is to be delivered **before** m_2
- **Total order broadcast**
  - Agreement on delivery order between the nodes
  - If m_1 is delivered before m_2 on one node, then m_1 must be delivered before m_2 on all nodes
- **FIFO-total order broadcast**
  - Combine FIFO and Total order broadcast

FIFO broadcast



- Valid delivery orders: $(m_2, m_1, m_3), (m_1, m_2, m_3), (m_1, m_3, m_2)$

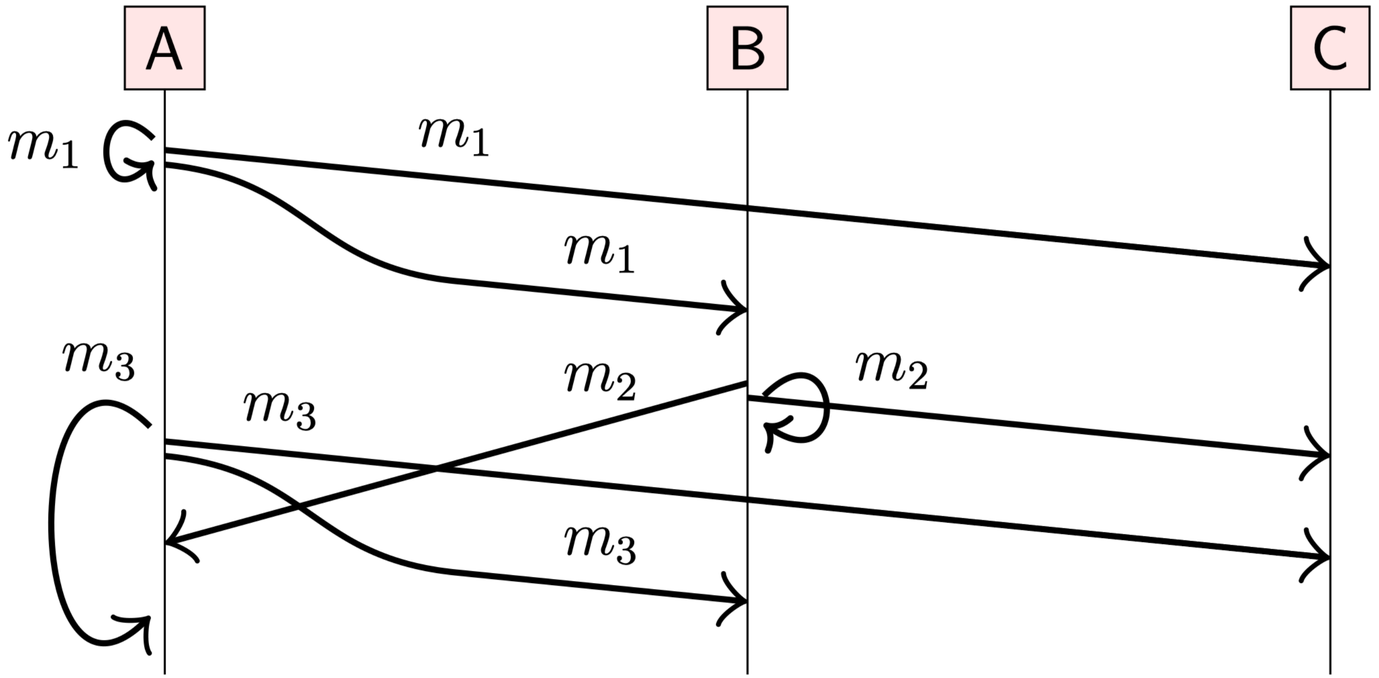
Causal broadcast



- We observe: broadcast(m_1) $\rightarrow$ broadcast(m_2) and broadcast(m_1) $\rightarrow$ broadcast(m_3)
- Valid orders are: $(m_1, m_2, m_3), (m_1, m_3, m_2)$

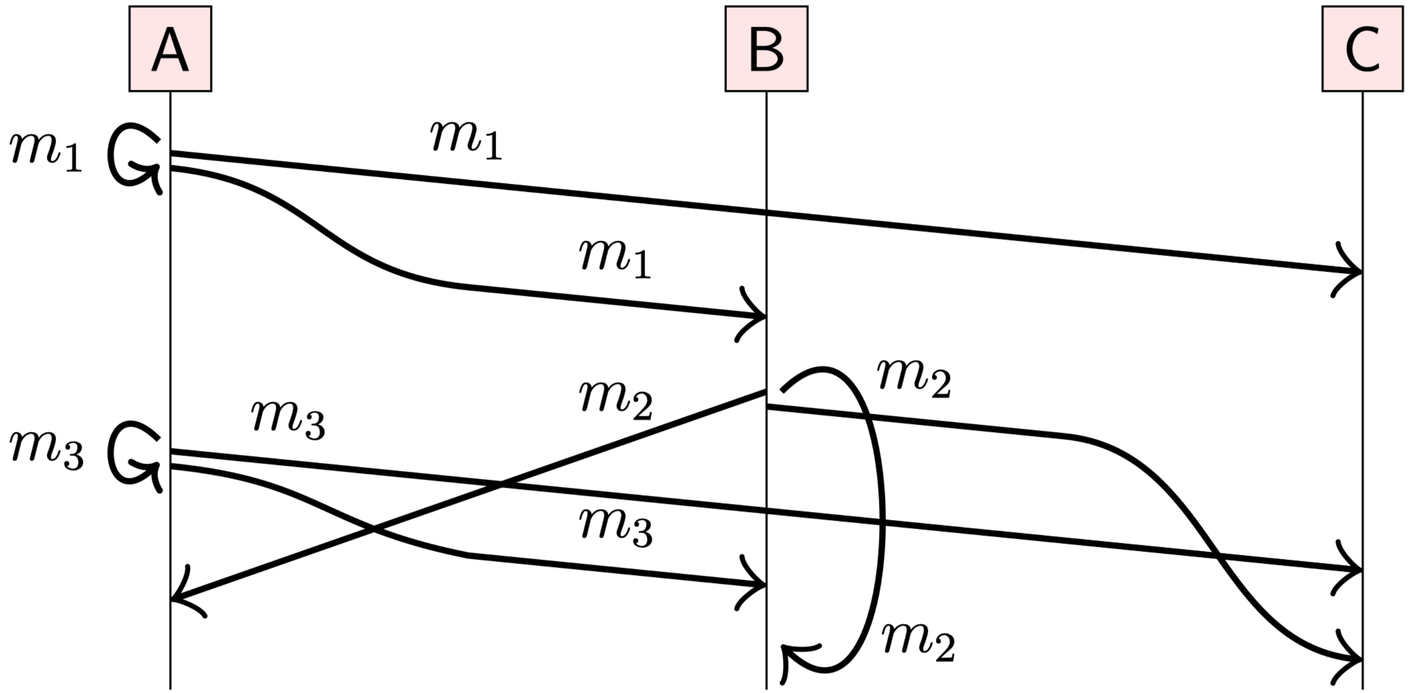
Total order broadcast

- All nodes must get the messages **in the same** order, e.g. m_1, m_2, m_3

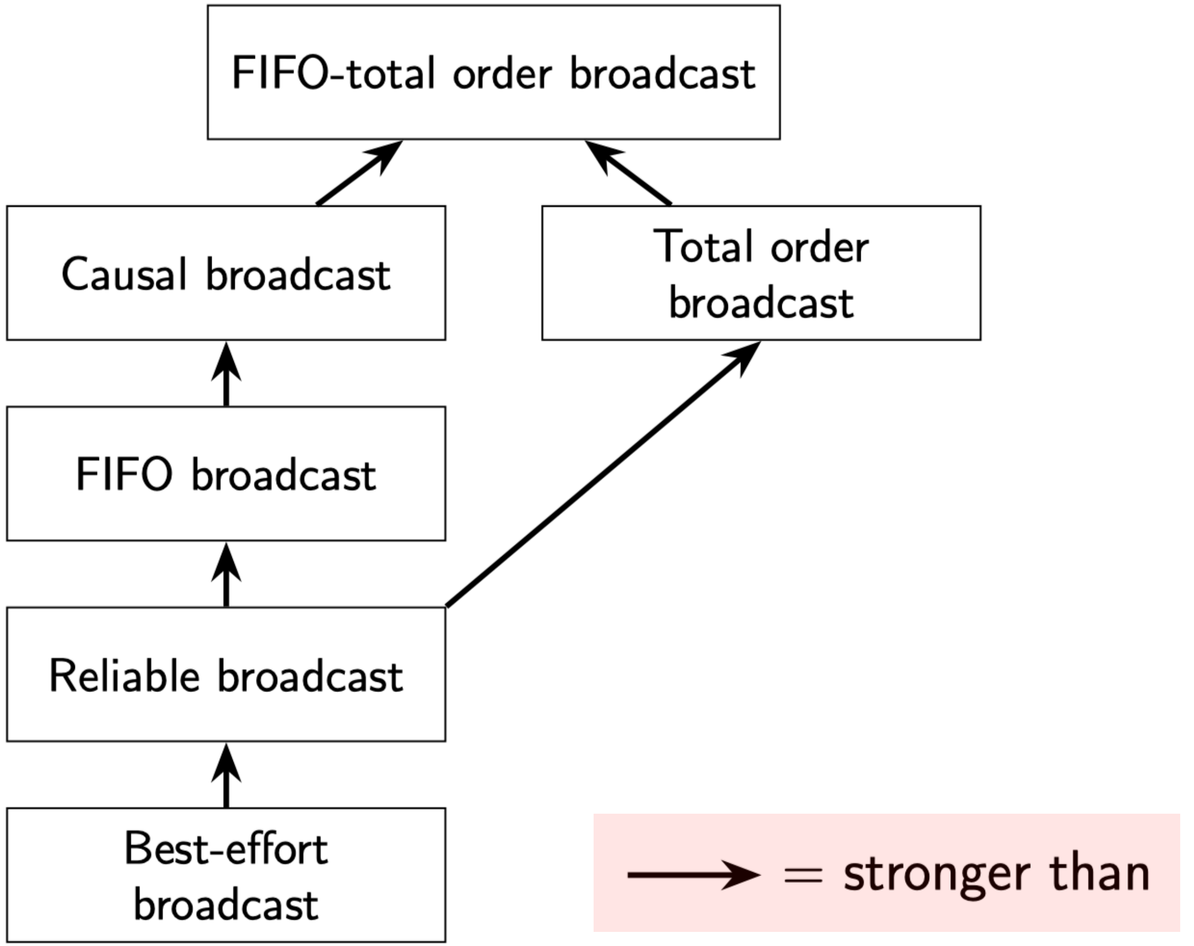


Total order broadcast

- Alternatively m_1, m_3, m_2

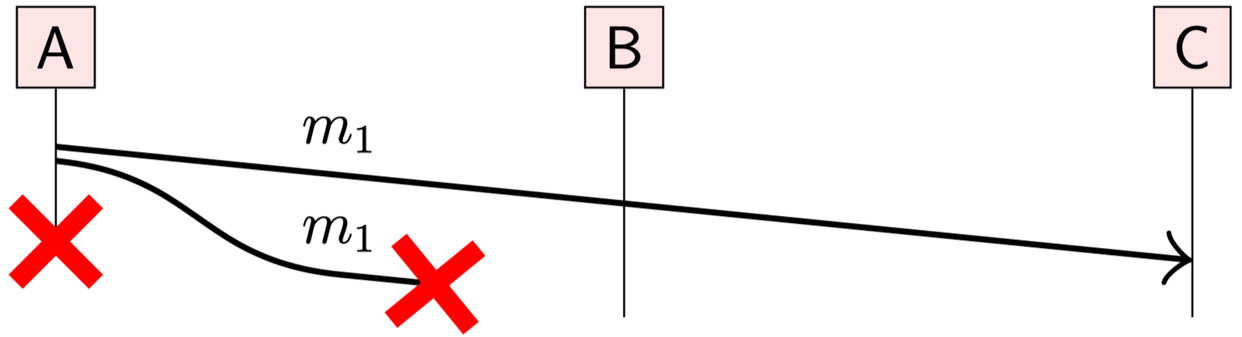


Relationships

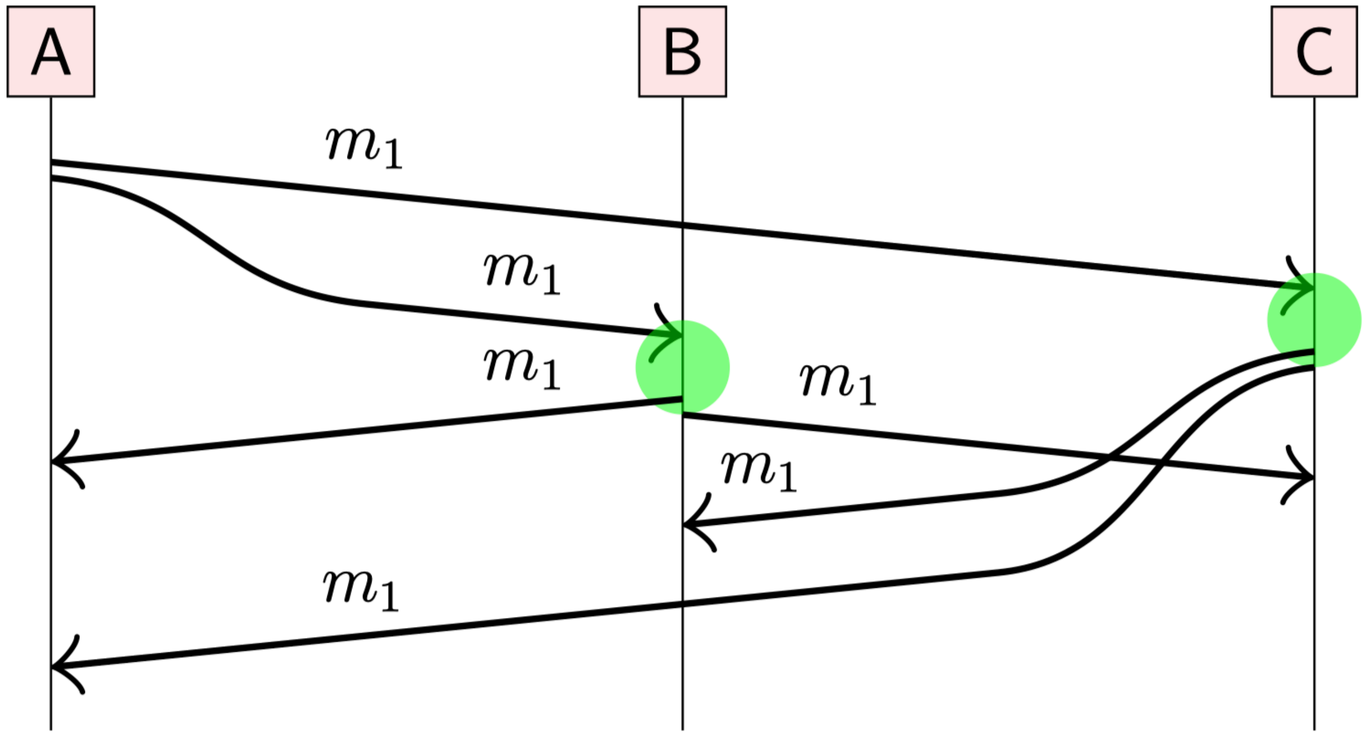


How to broadcast reliable

- Two goals:
  - Ensure that messages are delivered to all nodes' middleware (re-transmit etc.)
  - Deliver message to the app in the right node (by sorting in the middleware)
- Challenge: node & link crash



Eager reliable broadcast

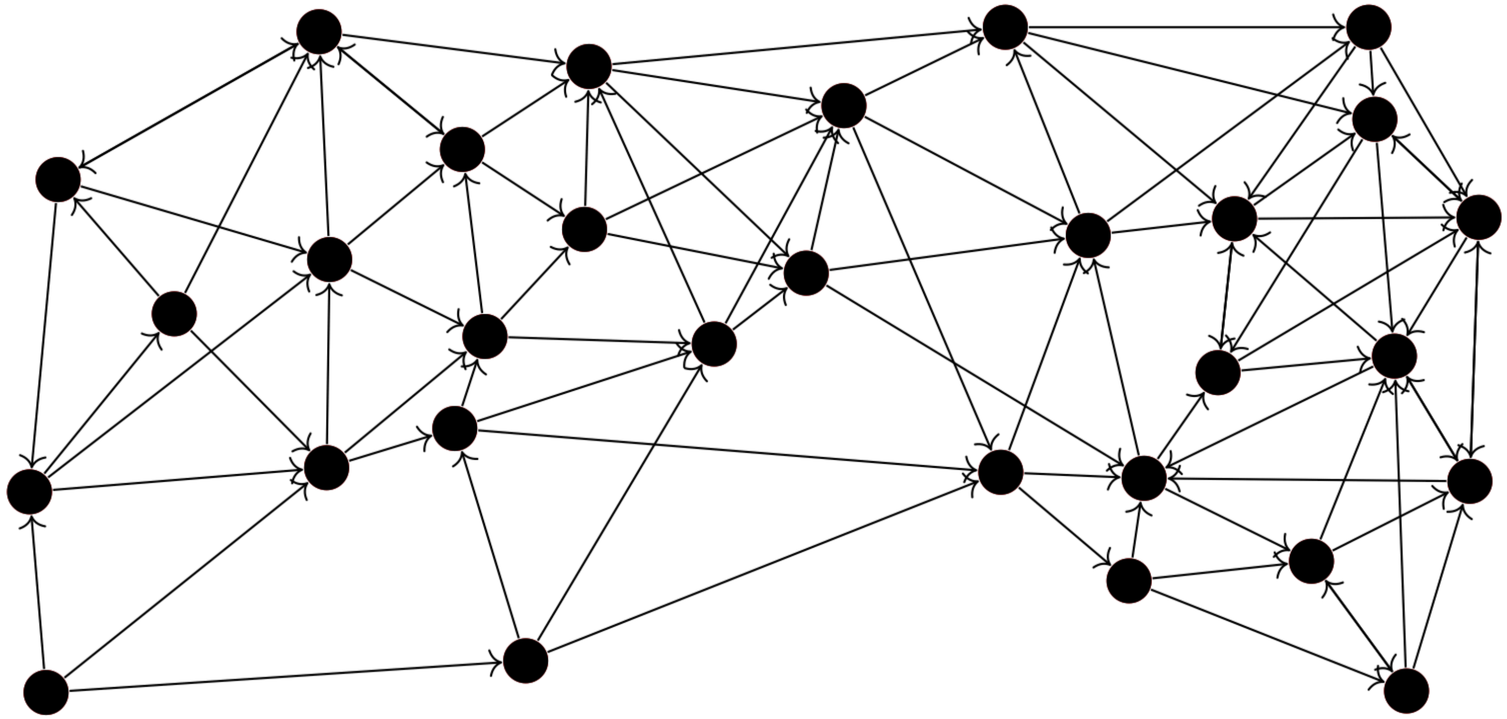


- The **first time** you receive a message, **re-broadcast** it to everyone

$O(n^2)$messages

Gossip protocol

- Choose 3 other random nodes to re-transmit
- Eventually the message will reach all nodes



FIFO broadcast algorithm

- Try to implement it in Python

on initialisation **do**
    $sendSeq := 0;\ delivered := \langle 0, 0, \ldots, 0 \rangle;\ buffer := \{\}$
end on

on request to broadcast m at node N_i **do**
    send $(i, sendSeq, m)$ via reliable broadcast
    $sendSeq := sendSeq + 1$
end on

on receiving msg from reliable broadcast at node N_i **do**
    $buffer := buffer \cup \{msg\}$
    while $\exists sender, m.\ (sender, delivered[sender], m) \in buffer$ **do**
        deliver m to the application
        $delivered[sender] := delivered[sender] + 1$
    end while
end on

```python
NODES = 3

buffer = {}
delivered = [0]*NODES

async def on_receive(reader, _):
    """
    Handle TCP message
    """

    while data := await reader.read(100):
        ldata = data.decode("utf8").split(",", maxsplit=2)
        sender, seq, msg = int(ldata[0]), int(ldata[1]), ldata[2].strip()

        print("Received:", sender, seq, msg)
        buffer[sender, seq] = msg

        def deliver_message():
            for n, x in enumerate(delivered):
                if (n, x) in buffer:
                    print(f"Delivering (node={n}, seq={x}):", buffer[n, x])
                    del buffer[n, x]
                    delivered[n] += 1
                    return True

        while deliver_message():
            pass
```



Causal broadcast

on initialisation **do**
$sendSeq := 0;\ delivered := \langle 0, 0, \ldots, 0 \rangle;\ buffer := \{\}$
end on

on request to broadcast m at node N_i **do**
$deps := delivered; deps[i] := sendSeq$
send $(i, deps, m)$ via reliable broadcast
$sendSeq := sendSeq + 1$
end on

on receiving msg from reliable broadcast at node N_i **do**
$buffer := buffer \cup \{msg\}$
while $\exists (sender, deps, m) \in buffer.\ deps \leq delivered$ **do**
deliver m to the application
$buffer := buffer \setminus \{(sender, deps, m)\}$
$delivered[sender] := delivered[sender] + 1$
end while
end on

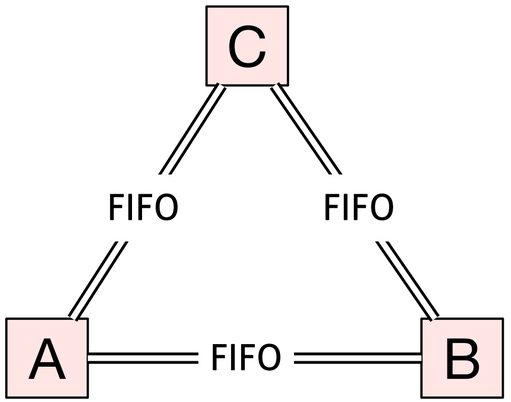


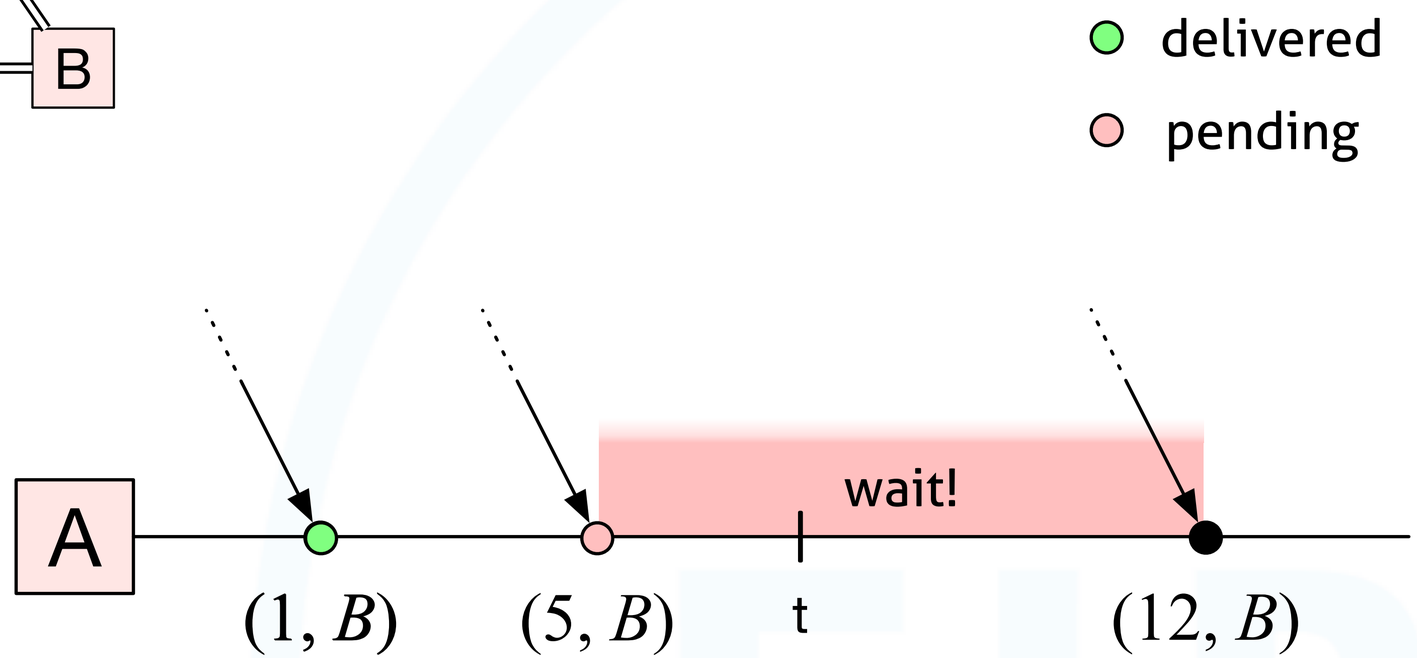
Homework!

Total order broadcast algorithm

- **Remember: all nodes receive the messages in the same order**
- Two usual approaches, but **not fault-tolerant**:
- **Single leader**
  - Designate a leader (called ***sequencer***) - "leader election"
  - To broadcast, send it to the **leader**, the leader will FIFO broadcast to the rest
  - Leader crash is **fatal**
- **Lamport clock** implementation:
  - Attach Lamport timestamp to every message
  - Deliver in the total order of timestamps
  - **Issue:** how do you know that you have seen all messages with a timestamp $t < T$? Need to use **FIFO links** and **wait** for messages with a timestamp $t \geq T$ from **every node**

Issue (continued)





Self-assesment

What is broadcasting?

Why do we need a middleware layer?

Compare FIFO and causal broadcast.

What is a total order broadcast?

How do we broadcast reliably?

Contrast FIFO and causal broadcast algorithms.

Why do we use gossip protocols?